# 3.5 Looping and Branching

# Looping in ARM

- Loop – repeating a sequence of instructions a certain number of times.
- e.g., To add 9 to r0 five times, I could do:

```
mov r0, #0
mov r1, #9
add r0, r0, r1
add r0, r0, r1
add r0, r0, r1
add r0, r0, r1
add r0, r0, r1
```

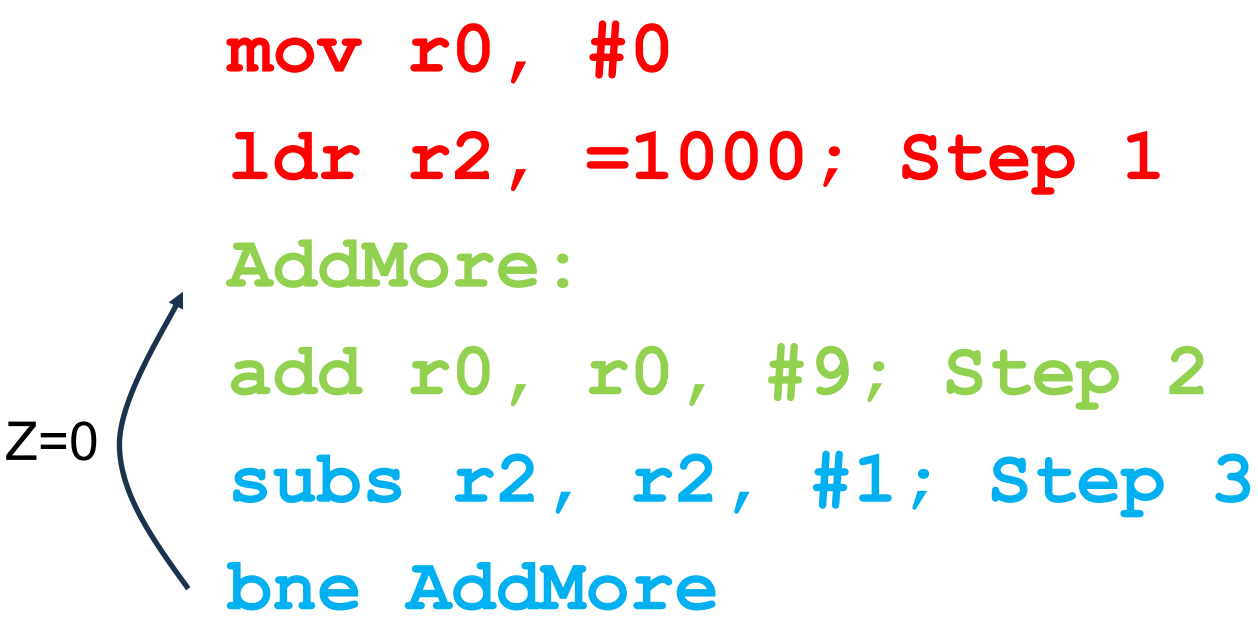
- Disadvantage: Too much code space is needed if repeating 100 times
- Use looping

# Three steps in writing a *for* loop

- Step 1: Initialize variables
  - Count: specifies how many times you want to repeat the loop. (e.g., [r2] = 5 in previous example)
  - Other variables depending on application.
- Step 2: Identify statement(s) to repeat
  - In the previous example, it would be:
    `add r0, r0, r1`
- Step 3: Determine whether to stop looping
  - Use `subs r2, r2, #1` to decrement [r2] after one iteration (repetition) of the loop, and check whether [r2] is 0. Two possibilities:
    - If [r2] = 0 (i.e., finished the desired number of iterations), exit loop.
    - If [r2] ≠ 0, branch back to Step 2.

# Example 1

- e.g., Write a program to (a) clear r0, (b) add 9 to r0 a thousand times.



# General format of a for loop

```
MOV   r0,#10    // Step 1
Loop:
code ...        // Step 2
SUBS  r0,r0,#1  // Step 3
BNE   Loop
Post loop ...   ;fall through on zero count
```

# Example 2

- Write a program to place the value 0x55 into 100 consecutive bytes starting from address 0x1500.
- Step 1: Initialization
  - [r1] = 0x5555555 (each time write 4 bytes)
  - [r0] = 25
  - [r2] = 0x1500
- Step 2: Statements to repeat
  - str r1, [r2], #4
- Step 3: Termination conditions
  - [r0] = [r0] – 1
  - Stop repetition if [r0] = 0

# Example 2

```
mov r0, #25
ldr r1, =0x55555555
ldr r2, =0x1500

Again:
str r1, [r2], #4
subs r0, r0, #1
bne Again

Here:
b Here
```

# While loop

- Pseudocode:

```
while expression
      statement(s)
end
```

- If *expression* is true, run *statement(s)* once.
- Assembly code:

```
Loop:
cmp r0, #0
beq WhileExit
…. statement(s)
b Loop
WhileExit:
Post loop ....; Exit
```

# Example 1 using while loop

- e.g., Write a program to (a) clear r0, (b) add 9 to r0 a thousand times.

```
mov r0, #0
ldr r2, =1000 // Step 1
AddMore:
cmp r2, #0
beq WhileExit
add r0, r0, #9 // Step 2
sub r2, r2, #1 // Step 3
b AddMore
WhileExit: Post loop; Exit
```

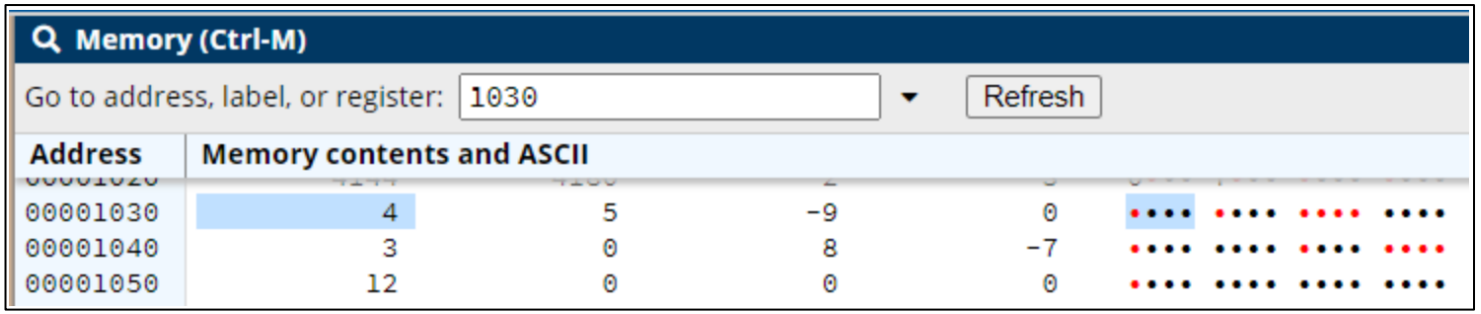
# Example on while loop from Lab 4

- This example traverses a list and operates on each value of the list.
- Let's have a good understanding of the list first:

```
.data  // Data section
Data:
.word    4,5,-9,0,3,0,8,-7,12    @ The list of data
_Data:    // End of list address
```

- The list begins at the address labeled *Data*
- The list values are comma-separated and are word-sized (as defined by *.word*)
- The address of the end of the list (i.e., immediately after the last item) is labeled as _*Data*. This is important as otherwise we have no way to know where the list ends.

# Example on while loop from Lab 4

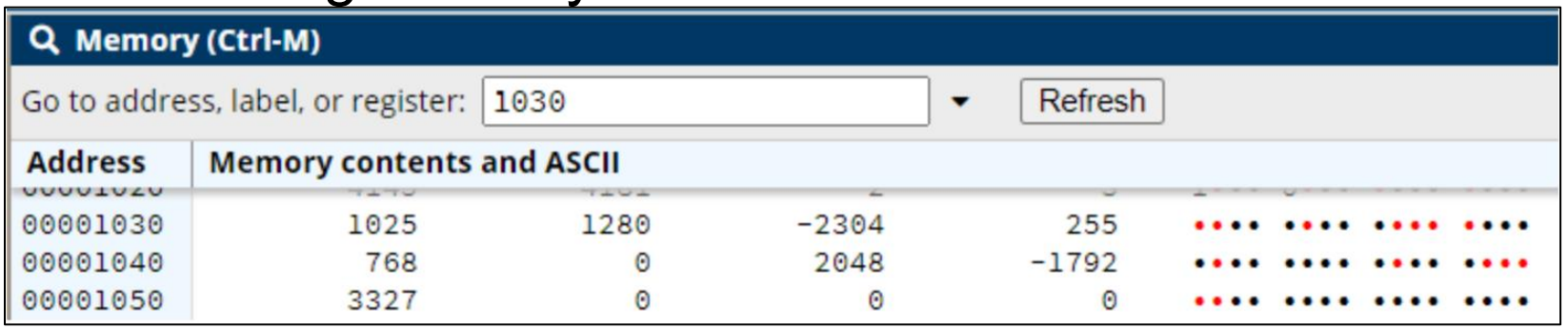



- Data items are stored correctly as displayed above because the list *Data* starts on the word boundary (address divisible by 4).
- However, this is not guaranteed. For example, the list would be unaligned by adding a single byte with a value of 1 in front of the *Data* list

```
1  .data
2  BAD:
3  .byte   1
4  Data:
5  .word   4,5,-9,0,3,0,8,-7,12    // The list of data
6  _Data:  // End of list address
```

# Example on while loop from Lab 4

The resulting memory looks like:




- The original list values are still there, but because they are now shifted by a byte, they no longer make sense in decimal.
- We can correct this with the *.align* directive:

```
1  .data
2  BAD:
3  .byte   1
4  .align  // Force list to start on a word boundary
5  Data:
6  .word   4,5,-9,0,3,0,8,-7,12    // The list of data
7  _Data:  // End of list address
```

*.align*: Fill address 1031 to 1033 by 0x00
Start writing the first word in address 1034

# Example on while loop from Lab 4

This code traverses the list:

```
16 .org 0x1000  // Start at memory location 1000
17 .text        // Code section
18 .global _start
19 _start:
20
21 ldr    r2, =Data    // Store address of start of list
22 ldr    r3, =_Data   // Store address of end of list
23
24 Loop:
25 ldr    r0, [r2], #4 // Read address with post-increment ([r0] <- [[r2]], [r2] <- [r2] + 4)
26 cmp    r3, r2       // Compare current address with end of list
27 bne    Loop         // If not at end, continue
28
29 _stop:
30 b _stop
31
32 .data
33 .align
34 Data:
35 .word   4,5,-9,0,3,0,8,-7,12    // The list of data
36 _Data: // End of list address
37
38 .end
```

# Example on while loop from Lab 4

```
1 ldr    r2, =Data    // Store address of start of list
2 ...
3 ldr    r0, [r2], #4 // Read address with post-increment ([r0] <- [[r2]], [r2] <- [r2] + 4)
```

does the following:

- loads the value at the address stored in r2 to r0. This is initially the first element of the *Data* list.
- The value of r2 is then incremented by 4. r2 now points to the second element of the *Data* list.

```
.data  // Data section
Data:
.word    4,5,-9,0,3,0,8,-7,12    @ The list of data
_Data:    // End of list address
```

- This continues until some end condition is reached.

# Example on while loop from Lab 4

The end condition in this case is to compare the current address in r3 against the address immediately following the end of the list, i.e., the value in *_Data*:

```
1 ldr r2, =Data      // Store address of start of list
2 ldr r3, =_Data     // Store address of end of list
3 …
4 Loop:
5 ldr    r0, [r2], #4 // Read address with post-increment ([r0] <- [[r2]], [r2] <- [r2] + 4)
6 …
7 cmp r3, r2         // Compare current address with end of list
8 bne Loop           // If not at end, continue
```

- If [r3]>[r2], end has not been reached, Z = 0, the branch bne is taken.
- If [r3]=[r2], end is reached, Z = 1, bne not taken.

# Example on while loop from Lab 4

- Suppose we want to process a list of values, and count how many positive, negative, and zero values are in the list.
- To do it:
  1. We would use three registers to store the number of positive, negative and zero values.
  2. For each value we traverse using the above program, we test whether it is greater than, equal to or less than 0 and add 1 to the appropriate register.

# Example on while loop from Lab 4

```
18 .text
19 ldr r2, =Data    // Store address of start of list
20 ldr r3, =_Data   // Store address of end of list
21 mov r5, #0       // Initialize negative count
22 mov r6, #0       // Initialize positive count
23 mov r7, #0       // Initialize zero count
24
25 Loop:
26 ldr r1, [r2], #4    // Read address with post-increment ([r0] <- [[r2]], [r2] <- [r2] + 4)
27
28 cmp r1, #0           // Compare number to 0
29 // Conditional executions
30 addlt   r5, r5, #1  // Increment the negative value count if r1 is negative
31 addgt   r6, r6, #1  // Increment the positive value count if r1 is positive
32 addeq   r7, r7, #1  // Increment the zero value count if r1 is zero
33
34 cmp     r2, r3      // Compare current address with end of list
35 bne     Loop        // If not at end, continue
```